Hash, HashMap and HashSet Study Guide

Hash

What is Hashing?

Hashing is a technique that converts any input (like a string or number) into a fixed-size value called a hash code. Think of it like a fingerprint - each piece of data gets a unique identifier that helps computers find and organize information quickly.

Hash Function

A hash function takes an input and produces a hash code (usually an integer). Java automatically generates hash codes for objects using the `hashCode()` method.

```java
String name = "Alice";
int hashValue = name.hashCode();
System.out.println("Hash code of 'Alice': " + hashValue);
// Output: Hash code of 'Alice': 63593391
```

Why Use Hashing?

- **Fast searching**: Finding data becomes much quicker
- **Efficient storage**: Data is organized in a way that makes access faster
- **Unique identification**: Each piece of data gets a unique identifier

HashMap

What is HashMap?

HashMap is a data structure that stores data in key-value pairs. It uses hashing to quickly find values based on their keys. Think of it like a dictionary where you look up a word (key) to find its meaning (value).

Creating a HashMap

```java
import java.util.HashMap;

// Create a HashMap
HashMap<String, Integer> ages = new HashMap<>();

// You can also specify initial capacity
HashMap<String, String> countries = new HashMap<>(16);
```

Basic HashMap Operations

Adding Elements (put)

```java
HashMap<String, Integer> studentGrades = new HashMap<>();

// Adding key-value pairs
studentGrades.put("Alice", 95);
studentGrades.put("Bob", 87);
studentGrades.put("Charlie", 92);

System.out.println(studentGrades);
// Output: {Alice=95, Bob=87, Charlie=92}
```

Getting Elements (get)

```java
// Get value by key
Integer aliceGrade = studentGrades.get("Alice");
System.out.println("Alice's grade: " + aliceGrade);
// Output: Alice's grade: 95

// Get value for non-existing key (returns null)
Integer davidGrade = studentGrades.get("David");
System.out.println("David's grade: " + davidGrade);
// Output: David's grade: null
```

Checking if Key or Value Exists

```java
// Check if key exists
boolean hasAlice = studentGrades.containsKey("Alice");
System.out.println("Has Alice: " + hasAlice); // Output: true

// Check if value exists
boolean hasGrade95 = studentGrades.containsValue(95);
System.out.println("Has grade 95: " + hasGrade95); // Output: true
```

Removing Elements

```java
// Remove by key
Integer removedGrade = studentGrades.remove("Bob");
System.out.println("Removed: " + removedGrade); // Output: 87

// Remove all elements
studentGrades.clear();
```

Size and Empty Check

```java
// Get size
int size = studentGrades.size();
System.out.println("Number of students: " + size);

// Check if empty
boolean isEmpty = studentGrades.isEmpty();
System.out.println("Is empty: " + isEmpty);
```

Iterating Through HashMap

```java
HashMap<String, Integer> fruits = new HashMap<>();
fruits.put("Apple", 5);
fruits.put("Banana", 3);
fruits.put("Orange", 8);

// Method 1: Using keySet()
for (String fruit : fruits.keySet()) {
    System.out.println(fruit + ": " + fruits.get(fruit));
}

// Method 2: Using entrySet()
for (HashMap.Entry<String, Integer> entry : fruits.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// Method 3: Using values() for values only
for (Integer quantity : fruits.values()) {
    System.out.println("Quantity: " + quantity);
}
```

HashMap Characteristics

- **No duplicate keys**: Each key can appear only once
- **Allows one null key**: You can have one null key
- **Allows multiple null values**: Values can be null
- **Not synchronized**: Not thread-safe
- **No ordering**: Elements are not stored in any particular order

```java
HashMap<String, String> example = new HashMap<>();
example.put(null, "null key value");  // Allowed
example.put("key1", null);            // Allowed
example.put("key2", null);            // Allowed - multiple null values
```

HashSet

What is HashSet?

HashSet is a collection that stores unique elements. It uses hashing internally to ensure fast operations and automatically prevents duplicate values. Think of it like a bag where you can't put the same item twice.

Creating a HashSet

```java
import java.util.HashSet;

// Create a HashSet
HashSet<String> colors = new HashSet<>();

// Create with initial capacity
HashSet<Integer> numbers = new HashSet<>(20);

// Create from another collection
HashSet<String> copyColors = new HashSet<>(colors);
```

Basic HashSet Operations

Adding Elements

```java
HashSet<String> animals = new HashSet<>();

// Add single elements
animals.add("Dog");
animals.add("Cat");
animals.add("Bird");
animals.add("Dog"); // Duplicate - won't be added

System.out.println(animals);
// Output: [Dog, Cat, Bird] (order may vary)
```

Removing Elements

```java
// Remove specific element
boolean removed = animals.remove("Cat");
System.out.println("Cat removed: " + removed); // Output: true

// Remove all elements
animals.clear();
```

Checking Elements

```java
HashSet<Integer> numbers = new HashSet<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);

// Check if element exists
boolean hasNumber = numbers.contains(20);
System.out.println("Contains 20: " + hasNumber); // Output: true

// Check size
int size = numbers.size();
System.out.println("Size: " + size); // Output: 3

// Check if empty
boolean isEmpty = numbers.isEmpty();
System.out.println("Is empty: " + isEmpty); // Output: false
```

Iterating Through HashSet

```java
HashSet<String> cities = new HashSet<>();
cities.add("New York");
cities.add("London");
cities.add("Tokyo");

// Method 1: Enhanced for loop
for (String city : cities) {
    System.out.println(city);
}

// Method 2: Using iterator
Iterator<String> iterator = cities.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

HashSet Characteristics

- **No duplicates**: Automatically prevents duplicate elements
- **Allows one null**: Can contain at most one null element
- **Not synchronized**: Not thread-safe
- **No ordering**: Elements are not stored in insertion order
- **Fast operations**: Add, remove, and contains operations are typically O(1)

```java
HashSet<String> example = new HashSet<>();
example.add("First");
example.add("First");    // Duplicate - ignored
example.add(null);       // Allowed
example.add(null);       // Second null - ignored

System.out.println(example); // Output: [First, null]
```

Converting Between HashSet and Arrays

```java
// Array to HashSet
String[] fruitArray = {"Apple", "Banana", "Apple", "Orange"};
HashSet<String> fruitSet = new HashSet<>(Arrays.asList(fruitArray));
System.out.println(fruitSet); // Duplicates removed

// HashSet to Array
String[] backToArray = fruitSet.toArray(new String[0]);
```

Differences Between HashMap and HashSet

Feature	HashMap	HashSet
Storage	Key-Value pairs	Single elements only
Duplicates	Duplicate values allowed, unique keys	No duplicates allowed
Null values	One null key, multiple null values	One null element
Access method	get(key)	contains(element)
Primary use	When you need key-value mapping	When you need unique collection

```java
// HashMap example
HashMap<String, Integer> studentAges = new HashMap<>();
studentAges.put("Alice", 20);
studentAges.put("Bob", 20);    // Same value, different key - OK

// HashSet example
HashSet<String> uniqueNames = new HashSet<>();
uniqueNames.add("Alice");
uniqueNames.add("Alice");      // Duplicate - ignored
```

Quiz Questions

1. What will be the output of the following code?

```java
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 1);
map.put("B", 2);
map.put("A", 3);
System.out.println(map.size());
```

a) 3

b) 2

c) 1

d) 4

Answer: b) 2

2. Which of the following is true about HashSet? a) It allows duplicate elements

b) It maintains insertion order

c) It can contain at most one null element

d) It stores key-value pairs

Answer: c) It can contain at most one null element

3. What method is used to check if a HashMap contains a specific key? a) hasKey()

b) containsKey()

c) checkKey()

d) findKey()

Answer: b) containsKey()

4. What will happen if you try to get a value from HashMap using a key that doesn't exist? a) Throws an exception

b) Returns an empty string

c) Returns null

d) Returns 0

Answer: c) Returns null

5. Which collection would be best for storing a list of unique student ID numbers? a) HashMap

b) HashSet

c) ArrayList

d) LinkedList

Answer: b) HashSet

Practice Coding Questions

1. Create a HashMap that stores student names as keys and their corresponding grades as values. Add at least 5 students with their grades, then print all students who have a grade above 85.

2. Write a program that takes an array of integers and uses a HashSet to find and print all duplicate numbers in the array.

3. Create a HashMap to count the frequency of each character in a given string. For example, for the string "hello", the output should show: h=1, e=1, l=2, o=1.

4. Using HashSet, write a program that finds the intersection (common elements) of two arrays of strings.

5. Create a program that uses HashMap to store country names as keys and their capitals as values. Implement methods to:

- Add a new country-capital pair
- Find the capital of a given country
- Check if a country exists in the map
- Display all countries and their capitals